

Lecture 1: Mathematical Foundations for Lattice-Based Cryptography

Groups, Rings, Fields, Modular Arithmetic,
Vector Spaces, Basis, and Lattices

Dr. Faisal Aslam

Lecture Notes — Session 1 (approx. 2 hours)

Purpose of this lecture. Every technical term used later in the thesis proposal — “lattice,” “modulus,” “ring $\mathbb{Z}_q[x]/(x^n + 1)$ ” — rests on a small set of algebraic building blocks. This lecture builds those blocks, one at a time, from the ground up: *Group* $\rightarrow$ *Subgroup* $\rightarrow$ *Ring* $\rightarrow$ *Field* $\rightarrow$ *Modular Arithmetic* $\rightarrow$ *Vector Space* $\rightarrow$ *Basis* $\rightarrow$ *Lattice*. Nothing here requires prior abstract algebra; every definition is followed immediately by concrete, worked examples.

Contents

1	Why Start With Algebra?	1
2	Sets and Binary Operations (Quick Refresher)	1
3	Groups	2
3.1	Subgroups	2
3.2	Cosets and Quotient Groups: Where $\mathbb{Z}_n$ Really Comes From	3
4	Rings	4
4.1	Zero Divisors: A Structural Reason Some Rings Aren’t Fields	5
5	Fields	6
6	The Modulus: Arithmetic That Wraps Around	8
6.1	Finding Inverses Without Guessing: The Extended Euclidean Algorithm	9
7	Vector Spaces, Basis, and Dimension	11
7.1	Inner Products, Norms, and Orthogonality	11
7.2	Gram–Schmidt Orthogonalization	12
8	Lattices, Properly Defined	13
8.1	Discrete Sets	13
8.2	The Definition	13
8.3	Recovering the Basis Picture	14
8.4	The Determinant (Volume) of a Lattice	16
8.5	Minkowski’s First Theorem: Volume Forces a Short Vector to Exist	16
9	Summary Table	20
10	Full List of Exercises (Assign as Homework Before Lecture 2)	21

1 Why Start With Algebra?

Lattice-based cryptography is, at its core, about doing arithmetic on *vectors* whose coordinates live in a *finite, wraparound number system*, arranged according to strict algebraic rules. Before we can even state what a lattice is, we need precise answers to questions like:

- What does it mean for a set of numbers to have a well-behaved notion of “add” and “multiply”? (**Groups, Rings, Fields**)
- What happens when arithmetic “wraps around,” like a clock? (**Modulus**)
- What does it mean to build every point in a space out of a small number of building-block directions? (**Vector spaces, Basis**)
- What happens when we only allow *whole-number* combinations of those building blocks, instead of any real-number combination? (**Lattice**)

This lecture answers each of these in order. By the end, the definition of a lattice (Section 8) should feel like the natural, inevitable next step, not an isolated new idea.

2 Sets and Binary Operations (Quick Refresher)

A **set** is just a collection of objects (numbers, vectors, symbols — anything). A **binary operation** on a set S is a rule that takes two elements of S and combines them into a third element, e.g., ordinary addition $+$ takes two numbers and produces their sum. We write $a * b$ for a general binary operation.

An operation is **closed** on S if combining two elements of S always gives another element of S (never something outside S). This sounds obvious but fails more often than you’d expect — see Example 1 below.

Example: Closure Can Fail

Let $S = \{1, 2, 3, 4, 5\}$ (just these five numbers) and let $*$ be ordinary addition. Then $2 * 4 = 6$, but $6 \notin S$. So ordinary addition is *not* closed on S . This is exactly the kind of problem modular arithmetic (Section 5) is built to fix.

3 Groups

Definition: Group

A set G together with a binary operation $*$ is called a **group**, written $(G, *)$, if all four of the following hold:

(G1) **Closure:** for all $a, b \in G$, $a * b \in G$.

(G2) **Associativity:** for all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.

(G3) **Identity:** there exists an element $e \in G$ such that $a * e = e * a = a$ for all $a \in G$.

(G4) **Inverses:** for every $a \in G$, there exists $a^{-1} \in G$ such that $a * a^{-1} = a^{-1} * a = e$.

If, in addition, $a * b = b * a$ for all $a, b \in G$ (the order doesn’t matter), the group is called **abelian** (commutative).

Example: Groups You Already Know

- $(\mathbb{Z}, +)$ — the integers under addition. Identity is 0; the inverse of a is $-a$. This is an abelian group.
- $(\mathbb{R} \setminus \{0\}, \times)$ — nonzero real numbers under multiplication. Identity is 1; the inverse of a is $1/a$. Also abelian. (We had to remove 0 because 0 has no multiplicative inverse.)
- $(\mathbb{Z}, \times)$ — integers under multiplication — is **not** a group: 2 has no inverse in $\mathbb{Z}$ (we'd need $1/2$, which isn't an integer). This fails axiom (G4).

Remark: Why Cryptography Cares About Groups

Many classical cryptographic schemes (Diffie–Hellman, ElGamal, elliptic-curve cryptography) are built directly on top of group structure — the security relies on certain problems (like “given g and g^a , find a ”) being hard inside a specific group. Lattice-based cryptography instead builds on richer structures (rings and modules, coming next), but the group is always the first layer underneath.

Exercise: Try This Before Next Lecture

Is $(\mathbb{Z}_5 \setminus \{0\}, \times \bmod 5) = (\{1, 2, 3, 4\}, \times \bmod 5)$ a group? Check all four axioms directly by building the multiplication table. (You'll need Section 6 for what “mod 5” means if it's new to you — come back to this after reading that section.)

3.1 Subgroups

There is one more group-related idea we need before we can properly define a lattice: a group living *inside* another group.

Definition: Subgroup

Let $(G, *)$ be a group. A subset $H \subseteq G$ is a **subgroup** of G if H is itself a group under the same operation $*$. In practice, this is easiest to check with a simple test: H is a subgroup of G if and only if (1) H is nonempty (usually: it contains the identity e), (2) H is closed under $*$ (for $a, b \in H$, $a * b \in H$), and (3) H is closed under inverses (for $a \in H$, $a^{-1} \in H$). Associativity is automatic, since it already held for all of G .

Example: Subgroups You Can Picture

- For any integer n , the set $n\mathbb{Z} = \{\dots, -2n, -n, 0, n, 2n, \dots\}$ (all multiples of n) is a subgroup of $(\mathbb{Z}, +)$: it contains 0, adding two multiples of n gives another multiple of n , and $-(kn)$ is also a multiple of n . For example, $2\mathbb{Z}$ is “all even integers.”
- The natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ are **not** a subgroup of $(\mathbb{Z}, +)$: $\mathbb{N}$ is closed under addition, but it fails closure under inverses — $1 \in \mathbb{N}$, but $-1 \notin \mathbb{N}$.

Looking Ahead: Why This Matters in Two Sections

Keep the example $n\mathbb{Z} \subseteq \mathbb{Z}$ in mind. In Section 8, we will define a lattice as a special kind of subgroup of $\mathbb{R}^n$ — and $n\mathbb{Z}$ is exactly a simple, one-dimensional lattice in disguise: an evenly-spaced, discrete set of points, closed under addition and negation, sitting inside the larger group $\mathbb{Z}$ (or $\mathbb{R}$).

3.2 Cosets and Quotient Groups: Where $\mathbb{Z}_n$ Really Comes From

Section 6 will introduce $\mathbb{Z}_n$ informally, as “remainders with wraparound arithmetic.” That’s the right intuition, but it’s worth seeing the fully rigorous construction now, because it is a direct, satisfying payoff of the subgroup idea — and it is the same construction (“take a group, collapse it by a subgroup”) that shows up again later when we build the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ that Kyber and Dilithium actually use.

Definition: Coset

Let H be a subgroup of an abelian group $(G, +)$. For $a \in G$, the **coset** $a+H$ is the set $\{a+h : h \in H\}$ — “shift H by a .” Two elements $a, b \in G$ turn out to produce the *same* coset, $a+H = b+H$, exactly when $a-b \in H$. This gives a clean way to sort all of G into non-overlapping “buckets,” each one a shifted copy of H .

Example: The Cosets of $3\mathbb{Z}$ in $\mathbb{Z}$

Take $G = \mathbb{Z}$ and $H = 3\mathbb{Z} = \{\dots, -3, 0, 3, 6, \dots\}$ (a subgroup, per the “Subgroups You Can Picture” example in Section 3.1). The cosets are:

- $0 + 3\mathbb{Z} = \{\dots, -3, 0, 3, 6, \dots\}$ (multiples of 3)
- $1 + 3\mathbb{Z} = \{\dots, -2, 1, 4, 7, \dots\}$ (remainder 1 when divided by 3)
- $2 + 3\mathbb{Z} = \{\dots, -1, 2, 5, 8, \dots\}$ (remainder 2 when divided by 3)

Every integer falls into exactly one of these three buckets — and which bucket it falls into is precisely its remainder mod 3. There is no fourth distinct coset: $3 + 3\mathbb{Z} = 0 + 3\mathbb{Z}$ (check: $3 - 0 = 3 \in 3\mathbb{Z}$), so the pattern repeats.

Definition: Quotient Group

The set of all distinct cosets of H in G , with addition defined by $(a+H) + (b+H) := (a+b)+H$, is itself a group — the **quotient group**, written G/H (“ $G \bmod H$ ”). Its identity element is the coset $0 + H = H$ itself.

Remark: $\mathbb{Z}_n$ Is Not Just Like $\mathbb{Z}/n\mathbb{Z}$ — It Is $\mathbb{Z}/n\mathbb{Z}$

Compare the coset example above to Section 6: the three cosets of $3\mathbb{Z}$ in $\mathbb{Z}$ correspond *exactly* to the three elements $\{0, 1, 2\}$ of $\mathbb{Z}_3$, and coset addition matches addition mod 3 exactly. This is not a coincidence or an analogy — it is a theorem: $\mathbb{Z}_n$, as defined informally in Section 6, is precisely the quotient group $\mathbb{Z}/n\mathbb{Z}$. “Wraparound arithmetic” and “sorting integers into cosets of $n\mathbb{Z}$ ” are two descriptions of the exact same object. This is the rigorous justification for a claim we made without proof in Section 6: that $\mathbb{Z}_n$ really is a well-defined group (in fact ring) under addition and multiplication with wraparound — it inherits this structure directly from being a quotient of $\mathbb{Z}$.

Exercise: Practice

Let $H = 4\mathbb{Z} \subseteq \mathbb{Z}$. List all distinct cosets of H in $\mathbb{Z}$, and confirm there are exactly 4 of them, matching $\mathbb{Z}_4 = \{0, 1, 2, 3\}$. Which coset does -5 belong to?

4 Rings

A ring is what you get when you ask a set to support *two* binary operations that play two structurally different roles — one behaves like addition (fully invertible), the other behaves like multiplication (weaker, no inverses required). We always *label* these two operations “+” and “×,” but — and this is worth being precise about, since it’s easy to misread — **the labels do not mean the operations have to literally be**

school-arithmetic addition and multiplication. They can be almost any two operations at all, as long as together they satisfy the axioms below. “+” and “×” are names for the *roles* the two operations play, not a requirement about what the operations must concretely do.

Definition: Ring

A set R with two binary operations $+$ and $\times$ is a **ring** $(R, +, \times)$ if:

(R1) $(R, +)$ is an abelian group (with identity element usually written 0).

(R2) $\times$ is associative: $(a \times b) \times c = a \times (b \times c)$.

(R3) The distributive laws hold: $a \times (b + c) = (a \times b) + (a \times c)$ and $(a + b) \times c = (a \times c) + (b \times c)$.

Note: a ring does *not* need multiplicative inverses, and multiplication need not be commutative in general (though in every ring we use in this thesis, it will be — these are called **commutative rings**). If there is an element $1 \in R$ with $a \times 1 = 1 \times a = a$ for all a , we say R is a **ring with unity**.

Remark: “+” and “×” Are Role Labels, Not a Restriction to Arithmetic

Compare this to groups (Section 3): a group has *one* operation, and we wrote it as a generic $*$ precisely to emphasize that it could be anything (addition, multiplication, matrix multiplication, function composition — whatever satisfies the axioms). A ring needs *two* operations, and it turns out we always call them “+” and “×” — but this is a naming convention for the two *roles*, not a claim that the operations must be ordinary numeric addition and multiplication. Concretely:

- **$n \times n$ matrices:** “+” is ordinary matrix addition (this forms an abelian group), and “×” is matrix *multiplication* (associative, distributes over +) — but matrix multiplication is **not** commutative in general, which is fine, since a ring’s axioms above don’t require it (only *commutative* rings require it, and this one wouldn’t qualify as one).
- **Boolean rings** (used in digital logic): “+” is XOR and “×” is AND on $\{0, 1\}$ — nothing to do with school arithmetic at all, yet every ring axiom above holds.

So whenever you meet a new ring in this course, the first question to ask is not “is + really addition and is × really multiplication?” but rather: *what are the two operations here, and do they satisfy axioms (R1)–(R3)?* The labels “+”/“×” just tell you which operation is playing which role once you’ve checked that.

Example: Rings You Already Know

- $(\mathbb{Z}, +, \times)$ — the integers form a commutative ring with unity 1. Notice 2 still has no multiplicative inverse, and that’s fine — rings don’t require one.
- **Polynomial rings**, e.g. $\mathbb{Z}[x]$: the set of all polynomials with integer coefficients (like $3x^2 - x + 7$), with ordinary polynomial addition and multiplication. This is a ring for exactly the same reason $\mathbb{Z}$ is.

Looking Ahead: The Ring That Kyber and Dilithium Actually Use

Both Kyber and Dilithium do their arithmetic inside a specific ring of the form

$$R_q = \mathbb{Z}_q[x]/(x^n + 1),$$

read as: “polynomials with coefficients taken modulo q (Section 6), where we also reduce powers of x using the rule $x^n \equiv -1$.” This looks intimidating right now, but it is built from exactly the pieces in this lecture: a polynomial ring (this section), with coefficients from $\mathbb{Z}_q$ (Section 6), further “folded” by a fixed rule. We will unpack this fully in a later lecture — for now, just notice that “ring” is not an abstract curiosity; it’s the literal object the cryptography is built on.

4.1 Zero Divisors: A Structural Reason Some Rings Aren't Fields

We will soon see (Section 5) that $\mathbb{Z}_n$ is a field exactly when n is prime. Section 6.1 will justify this computationally, via gcd. Here is the deeper, structural reason *why* — worth seeing now, while we're still inside general ring theory.

Definition: Zero Divisor

In a ring R , a nonzero element a is a **zero divisor** if there exists some *nonzero* $b \in R$ with $a \times b = 0$. A commutative ring with unity that has *no* zero divisors (other than needing a or b to be 0) is called an **integral domain**.

Example: Zero Divisors in $\mathbb{Z}_6$, Absent in $\mathbb{Z}_5$

In $\mathbb{Z}_6$: $2 \times 3 = 6 \equiv 0 \pmod{6}$, but neither 2 nor 3 is 0. So 2 and 3 are both zero divisors — $\mathbb{Z}_6$ is *not* an integral domain. Contrast this with $\mathbb{Z}_5$: checking every pair of nonzero elements (1, 2, 3, 4), no product is ever $\equiv 0 \pmod{5}$ (e.g., $2 \times 3 = 6 \equiv 1$, $4 \times 4 = 16 \equiv 1$, etc.) — $\mathbb{Z}_5$ has no zero divisors.

Remark: Why This Explains the Prime/Composite Split

Fact: every field is automatically an integral domain (if $a \neq 0$ and $ab = 0$, multiply both sides by a^{-1} to get $b = 0$ — so no nonzero b can pair with a nonzero a to give 0). Contrapositive: *if a ring has a zero divisor, it cannot be a field*. Now, $\mathbb{Z}_n$ has a zero divisor exactly when n is composite: write $n = jk$ with $1 < j, k < n$; then $j \times k = n \equiv 0 \pmod{n}$, with neither j nor k equal to 0 in $\mathbb{Z}_n$. So every composite modulus produces zero divisors, which immediately rules out being a field — a purely structural explanation, with no gcd computation needed, of the same fact Section 6.1 reaches computationally.

Exercise: Practice

Find a pair of zero divisors in $\mathbb{Z}_{12}$ (there are several valid answers). Then explain, using the Fact above, why $\mathbb{Z}_{12}$ cannot be a field — without computing any inverses directly.

5 Fields

Definition: Field

A **field** is a set F with two binary operations $+$ and $\times$ such that:

(F1) $(F, +)$ is an abelian group, over *all* of F (identity element 0).

(F2) $(F \setminus \{0\}, \times)$ is an abelian group, over F **with 0 removed** (identity element 1).

(F3) The distributive law holds: $a \times (b + c) = (a \times b) + (a \times c)$.

Informally: a field is a set where you can add, subtract, multiply, and divide (by anything except 0) and always stay inside the set. In short — **a field is two abelian groups sharing one set, glued together by distributivity.**

Remark: Why 0 Has to Be Excluded, and It's Not Optional

In *any* structure with a distributive law, $0 \times x = 0$ for every x (this follows from distributivity itself, not from an extra rule: $0 \times x = (0 + 0) \times x = 0 \times x + 0 \times x$, so subtracting $0 \times x$ from both sides gives $0 = 0 \times x$). So 0 could only have a multiplicative inverse if $0 \times x = 1$ for some x — but the left side is always 0, never 1 (as long as $1 \neq 0$, which we always assume). So 0 is permanently excluded from having an inverse, in every field, without exception; it isn't a special case someone chose, it's forced by the arithmetic.

Definition: Subtraction and Division Are *Derived*, Not Extra

Neither subtraction nor division is a new, independent operation requiring its own axiom — both are just notation for “combine with an inverse,” one level for each operation:

$$a - b := a + (-b), \quad \frac{a}{b} := a \times b^{-1} \quad (b \neq 0),$$

where $-b$ is guaranteed to exist by $(F, +)$ being a group (F1), and b^{-1} is guaranteed to exist, for every *nonzero* b , by $(F \setminus \{0\}, \times)$ being a group (F2). So “a field supports division” is not an extra requirement on top of the definition — it falls straight out of (F2) once every nonzero element is already promised an inverse. This is exactly why (F2) excludes 0: since 0 can never have an inverse (see the remark above), “dividing by 0” can never be defined this way, no matter what field you're in.

Example: Division in $\mathbb{Z}_5$, Worked Out

What is $4 \div 3$ in $\mathbb{Z}_5$? First find $3^{-1} \bmod 5$: $3 \times 2 = 6 \equiv 1 \pmod{5}$, so $3^{-1} = 2$. Then $4 \div 3 := 4 \times 3^{-1} = 4 \times 2 = 8 \equiv 3 \pmod{5}$. Check: $3 \times 3 = 9 \equiv 4 \pmod{5}$ — correct, exactly as “ $4 \div 3 = 3$ ” should verify.

Remark: The One-Line Takeaway

Subtraction is the additive inverse; division is the multiplicative inverse. Same trick, applied once per operation — $a - b$ means “add a and b 's additive inverse,” a/b means “multiply a by b 's multiplicative inverse.” The one asymmetry between them traces directly back to (F1) vs. (F2): *every* element of F has an additive inverse, since $(F, +)$ is a group over *all* of F — so subtraction always works. But *only nonzero* elements have a multiplicative inverse, since $(F \setminus \{0\}, \times)$ is a group only after removing 0 — so division by b only ever works when $b \neq 0$.

Fact: Every Field Is a Ring

Every field F (Definition above) is automatically a commutative ring with unity — in fact, an **integral domain** (Section 4.1). *Why*: (F1) is exactly ring axiom (R1). Associativity and distributivity of $\times$ (ring axioms (R2)–(R3)) are inherited directly from $\times$ being a group operation on $F \setminus \{0\}$ together with (F3). So a field satisfies every ring axiom, plus two extra guarantees stacked on top: $\times$ is commutative, and every *nonzero* element is invertible. This is why some textbooks instead *define* a field as “a commutative ring with unity in which every nonzero element has a multiplicative inverse” — that phrasing and the definition above pick out exactly the same objects; it's just built in the reverse order (starting from “ring” and adding a condition, instead of starting from two groups and gluing them together). Finally, a field is automatically an integral domain too: if $ab = 0$ with $a \neq 0$, multiplying both sides by a^{-1} forces $b = a^{-1}(ab) = a^{-1} \cdot 0 = 0$, so there can be no zero divisors.

Example: Fields and Non-Fields

- $\mathbb{Q}$ (rationals), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex numbers) are all fields — you can divide by any nonzero element and stay inside the set.
- $\mathbb{Z}$ is **not** a field: $2 \in \mathbb{Z}$ has no multiplicative inverse in $\mathbb{Z}$ (again, $1/2 \notin \mathbb{Z}$). $\mathbb{Z}$ is a ring, but not a field.
- $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$ under addition and multiplication mod 5 *is* a field — every nonzero element has an inverse (you checked this, or will check this, in the Section 3 subgroup/group exercise). Fields built this way, from $\mathbb{Z}_p$ with p prime, are called **finite fields** or **Galois fields**, written $\mathbb{F}_p$ or $GF(p)$.

Remark: The One-Sentence Summary So Far

Group $\subset$ Ring $\subset$ Field, in the sense that a field is a ring with extra structure (division), and a ring is built from a group with extra structure (a second operation). Each step adds a capability: groups let you undo one operation (inverses); rings add a second, distributive operation; fields guarantee *both* operations are fully invertible (except division by 0).

6 The Modulus: Arithmetic That Wraps Around

Definition: Congruence Modulo n

For a fixed positive integer n (the **modulus**), two integers a and b are **congruent modulo n** , written

$$a \equiv b \pmod{n},$$

if n divides $(a - b)$ evenly — equivalently, a and b leave the same remainder when divided by n . The set of possible remainders, $\{0, 1, 2, \dots, n - 1\}$, together with addition and multiplication performed “with wraparound,” is written $\mathbb{Z}_n$ (or $\mathbb{Z}/n\mathbb{Z}$).

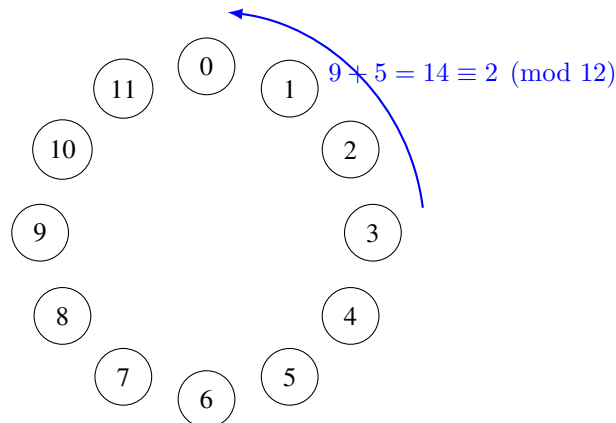


Figure 1: The clock analogy for modular arithmetic: $\mathbb{Z}_{12}$ works exactly like a 12-hour clock face. 9 o'clock plus 5 hours wraps around to 2 o'clock, i.e., $9 + 5 = 14 \equiv 2 \pmod{12}$.

Example: Computing Modulo

$17 \bmod 5$: divide 17 by 5 to get 3 remainder 2, so $17 \equiv 2 \pmod{5}$. Similarly, $23 \equiv 3 \pmod{10}$, and $-1 \equiv n - 1 \pmod{n}$ (e.g., $-1 \equiv 4 \pmod{5}$) — negative numbers wrap around too.

Remark: When Is $\mathbb{Z}_n$ a Ring? When Is It a Field?

$\mathbb{Z}_n$ is *always* a commutative ring with unity, for any $n \geq 2$. But it is a *field* only when $n = p$ is **prime**. If n is composite (e.g., $n = 6$), some nonzero elements fail to have multiplicative inverses — e.g., in $\mathbb{Z}_6$, there is no x with $2x \equiv 1 \pmod{6}$, since 2 shares a common factor with 6. This is exactly why cryptographic schemes that need a field (rather than just a ring) are built with a prime modulus q . (Section 4.1 gave the deeper structural reason: composite n always produces zero divisors, and a ring with zero divisors can never be a field.)

6.1 Finding Inverses Without Guessing: The Extended Euclidean Algorithm

So far, the only way we’ve found a multiplicative inverse mod n is trial and error (Exercise: “find x with $3x \equiv 1 \pmod{7}$ ” — you could just try $x = 1, 2, 3, \dots$ until one works). That’s fine for small numbers, but real cryptographic moduli are enormous (Kyber’s modulus is $q = 3329$; RSA moduli have hundreds of digits) — trial and error is completely impractical. We need an actual algorithm.

Definition: Greatest Common Divisor (GCD)

For integers a, b (not both zero), $\gcd(a, b)$ is the largest positive integer that divides both a and b . Two integers are **coprime** (or relatively prime) if $\gcd(a, b) = 1$.

Remark: Fact: When Does an Inverse Exist?

An element $a \in \mathbb{Z}_n$ has a multiplicative inverse if and only if $\gcd(a, n) = 1$. This is why every nonzero element of $\mathbb{Z}_p$ (for prime p) has an inverse: any a with $1 \leq a \leq p - 1$ automatically satisfies $\gcd(a, p) = 1$, since p ’s only divisors are 1 and p itself.

The classical **Euclidean Algorithm** computes $\gcd(a, b)$ by repeated division with remainder: $\gcd(a, b) = \gcd(b, a \bmod b)$, stopping when the remainder hits 0. The **Extended Euclidean Algorithm** does this same process but also tracks, at *every single step*, how the current remainder can be written as a combination $a \cdot x + b \cdot y$ (integers x, y) — and when the algorithm finishes with $\gcd(a, b) = 1$, the row where the remainder equals 1 hands you the inverse directly. The cleanest way to track this — much easier to follow than plain back-substitution — is a running **table** with one row per step.

Definition: The Extended Euclidean Algorithm, as a Table

To find $\gcd(a, b)$ together with integers x, y satisfying $a \cdot x + b \cdot y = \gcd(a, b)$, build a table with columns q (quotient), r (remainder), x, y , starting with two fixed initial rows and then one new row per division step:

step	q	r	x	y
−1	—	a	1	0
0	—	b	0	1
1	$q_1 = \lfloor r_{-1}/r_0 \rfloor$	$r_1 = r_{-1} - q_1 r_0$	$x_1 = x_{-1} - q_1 x_0$	$y_1 = y_{-1} - q_1 y_0$
2	$q_2 = \lfloor r_0/r_1 \rfloor$	$r_2 = r_0 - q_2 r_1$	$x_2 = x_0 - q_2 x_1$	$y_2 = y_0 - q_2 y_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Every single row satisfies the **invariant** $r_i = a \cdot x_i + b \cdot y_i$ by construction — this is the whole mechanism, and it’s worth checking explicitly on at least one row the first time you use this table, to convince yourself it isn’t magic. Stop once a row produces $r_i = 0$; the *previous* row’s r is $\gcd(a, b)$, and that row’s x, y are exactly the combination you want.

Example: Table Method, Short Version: $\gcd(7, 3)$ and $3^{-1} \bmod 7$

step	q	r	x	y	check: $7x + 3y$
-1	—	7	1	0	$7(1) + 3(0) = 7 \checkmark$
0	—	3	0	1	$7(0) + 3(1) = 3 \checkmark$
1	$\lfloor 7/3 \rfloor = 2$	$7 - 2(3) = 1$	$1 - 2(0) = 1$	$0 - 2(1) = -2$	$7(1) + 3(-2) = 1 \checkmark$
2	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$0 - 3(1) = -3$	$1 - 3(-2) = 7$	(stop: $r = 0$)

The row *before* r hit 0 is step 1: $r_1 = 1 = \gcd(7, 3)$, with $x_1 = 1, y_1 = -2$. So $7(1) + 3(-2) = 1$, i.e., $3 \times (-2) \equiv 1 \pmod{7}$, giving $3^{-1} \equiv -2 \equiv 5 \pmod{7}$ — matching what trial and error would also find, but now via a method with a clear, mechanical stopping point.

Example: Table Method, Longer Version: $15^{-1} \bmod 26$

Here the table earns its keep — more steps than the tiny example above, but the process is identical, row by row.

step	q	r	x	y
-1	—	26	1	0
0	—	15	0	1
1	$\lfloor 26/15 \rfloor = 1$	$26 - 1(15) = 11$	$1 - 1(0) = 1$	$0 - 1(1) = -1$
2	$\lfloor 15/11 \rfloor = 1$	$15 - 1(11) = 4$	$0 - 1(1) = -1$	$1 - 1(-1) = 2$
3	$\lfloor 11/4 \rfloor = 2$	$11 - 2(4) = 3$	$1 - 2(-1) = 3$	$-1 - 2(2) = -5$
4	$\lfloor 4/3 \rfloor = 1$	$4 - 1(3) = 1$	$-1 - 1(3) = -4$	$2 - 1(-5) = 7$
5	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$3 - 3(-4) = 15$	$-5 - 3(7) = -26$

The row before r hit 0 is step 4: $r_4 = 1 = \gcd(26, 15)$, with $x_4 = -4, y_4 = 7$. **Check the invariant directly:** $26(-4) + 15(7) = -104 + 105 = 1 \checkmark$. Reading this modulo 26: $15 \times 7 \equiv 1 \pmod{26}$, so $15^{-1} \equiv 7 \pmod{26}$. **Final check:** $15 \times 7 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26}$. Correct.

Exercise: Practice

Use the table method above (not back-substitution, and not trial and error) to find $5^{-1} \bmod 11$. Build the full table — steps $-1, 0, 1, 2, \dots$ — and stop at the correct row. Then check your answer by direct multiplication.

Looking Ahead: Why Cryptography Loves the Modulus

Modular arithmetic gives us two things cryptography needs badly: (1) a **finite** number system (so keys and computations have a fixed, bounded size, no matter how much you compute), and (2) built-in **one-wayness** in places — e.g., given only $a \bmod n$, you cannot tell which of infinitely many integers a actually was. The Learning-With-Errors (LWE) problem from the proposal document hides its secret inside equations that live in exactly this kind of finite, modular number system (there, the modulus is usually called q).

Exercise: Practice

(a) Compute $23 \times 15 \bmod 7$. (b) List all elements of $\mathbb{Z}_8$ that *do not* have a multiplicative inverse, and explain why (hint: compare each element to 8). (c) Confirm $\mathbb{Z}_7$ is a field by checking that 3 has a multiplicative inverse mod 7 (find x such that $3x \equiv 1 \pmod{7}$).

7 Vector Spaces, Basis, and Dimension

Definition: Vector Space

A **vector space** over a field F is a set V (whose elements are called **vectors**) equipped with an addition operation $V \times V \rightarrow V$ and a scalar multiplication operation $F \times V \rightarrow V$, satisfying, for all $u, v, w \in V$ and all $a, b \in F$:

(V1) **Closure of addition:** $u + v \in V$.

(V2) **Associativity of addition:** $(u + v) + w = u + (v + w)$.

(V3) **Commutativity of addition:** $u + v = v + u$.

(V4) **Additive identity:** there exists $\vec{0} \in V$ with $v + \vec{0} = v$ for all v .

(V5) **Additive inverses:** for every $v \in V$, there exists $-v \in V$ with $v + (-v) = \vec{0}$.

(V6) **Compatibility of scalar multiplication with field multiplication:** $a(bv) = (ab)v$.

(V7) **Identity element of scalar multiplication:** $1v = v$, where 1 is the multiplicative identity of F .

(V8) **Distributivity, two directions:** $a(u + v) = au + av$ and $(a + b)v = av + bv$.

Axioms (V1)–(V5) say exactly “ $(V, +)$ is an abelian group” (Section 3) — nothing new there. Axioms (V6)–(V8) are the genuinely new content: they say scalar multiplication by field elements behaves consistently with the field’s own addition and multiplication (Sections 4–5). The most important example for us: $\mathbb{R}^n$, the set of all n -tuples of real numbers, is a vector space over $\mathbb{R}$ — and it’s a good exercise to mentally check each of (V1)–(V8) holds there before moving on.

Definition: Linear Combination, Span, Linear Independence

Given vectors $v_1, \dots, v_k \in V$ and scalars $c_1, \dots, c_k \in F$, the vector $c_1v_1 + c_2v_2 + \dots + c_kv_k$ is called a **linear combination** of $v_1, \dots, v_k$. The set of *all* linear combinations of a set of vectors is called their **span**. The vectors $v_1, \dots, v_k$ are **linearly independent** if the only way to write $0 = c_1v_1 + \dots + c_kv_k$ is with every $c_i = 0$ — informally, no vector in the set is “redundant” (expressible using the others).

Definition: Basis and Dimension

A set of vectors $B = \{b_1, \dots, b_k\}$ is a **basis** of a vector space V if (1) B is linearly independent, and (2) B **spans** V (every vector in V can be written as a linear combination of B). Every basis of a given vector space has the same number of elements k , called the **dimension** of V .

Example: Basis of $\mathbb{R}^2$

The vectors $e_1 = (1, 0)$ and $e_2 = (0, 1)$ form the “standard” basis of $\mathbb{R}^2$: any vector (a, b) is exactly $a \cdot e_1 + b \cdot e_2$. But this is far from the *only* basis — $\{(1, 1), (1, -1)\}$ is also a valid basis of $\mathbb{R}^2$ (check: they’re linearly independent, and any (a, b) can be written as a combination of them). A vector space almost always has infinitely many possible bases; some are more convenient to work with than others. **Keep this fact in mind — it becomes very important in Section 8.**

7.1 Inner Products, Norms, and Orthogonality

To make “good basis” (short, nearly-perpendicular vectors) versus “bad basis” (long, nearly-parallel vectors) precise rather than just intuitive, we need a way to measure length and angle.

Definition: Inner Product and Norm

For $x = (x_1, \dots, x_n), y = (y_1, \dots, y_n) \in \mathbb{R}^n$, the (standard) **inner product** is $\langle x, y \rangle = \sum_{i=1}^n x_i y_i$. The **norm** (length) of x is $\|x\| = \sqrt{\langle x, x \rangle}$. Two vectors are **orthogonal** if $\langle x, y \rangle = 0$ (the angle between them is 90°). A basis $\{b_1, \dots, b_k\}$ is an **orthogonal basis** if every pair b_i, b_j ($i \neq j$) is orthogonal.

Example: Computing Norms and Checking Orthogonality

For $x = (3, 4)$: $\|x\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$. For $x = (1, 1)$ and $y = (1, -1)$: $\langle x, y \rangle = (1)(1) + (1)(-1) = 0$, so these two vectors (the alternate basis of $\mathbb{R}^2$ from the previous example) are orthogonal — a nicer basis, geometrically, than it might have looked at first glance.

Looking Ahead: Quantifying “Good” vs. “Bad” Bases

With norms in hand, we can finally give a real, measurable meaning to the “good basis / bad basis” idea flagged repeatedly in this lecture: a good basis has vectors with small norm $\|b_i\|$ that are close to orthogonal; a bad basis for the *same* lattice can have arbitrarily large norms and near-parallel vectors, despite generating an identical set of points. One common numerical measure of “how good” a basis is is its **orthogonality defect**, $\frac{\prod_i \|b_i\|}{\det(L)}$ (see Section 8.4 below for $\det(L)$) — it equals 1 for a perfectly orthogonal basis and grows the more skewed the basis is.

7.2 Gram–Schmidt Orthogonalization

Given *any* basis, can we systematically produce an orthogonal one (spanning the same space, though generally *not* the same lattice — more on that distinction below)? Yes:

Definition: Gram–Schmidt Process

Given linearly independent $b_1, \dots, b_k$, define $b_1^* = b_1$, and for $i = 2, \dots, k$:

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{i,j} b_j^*, \quad \text{where } \mu_{i,j} = \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}.$$

Each b_i^* is b_i with the “shadow” it casts on all previous b_j^* subtracted off, leaving only the genuinely new, orthogonal direction. The result $\{b_1^*, \dots, b_k^*\}$ is an orthogonal basis of the same vector space (though, importantly, its integer combinations generally do *not* produce the same lattice as the original b_i ’s — Gram–Schmidt vectors are a vector-space tool, not automatically a lattice basis).

Example: Gram–Schmidt on a 2×2 Example

Let $b_1 = (2, 0)$, $b_2 = (1, 2)$. Then $b_1^* = b_1 = (2, 0)$. Compute $\mu_{2,1} = \frac{\langle b_2, b_1^* \rangle}{\langle b_1^*, b_1^* \rangle} = \frac{(1)(2) + (2)(0)}{4} = \frac{2}{4} = 0.5$. So $b_2^* = b_2 - 0.5 b_1^* = (1, 2) - (1, 0) = (0, 2)$. Check: $\langle b_1^*, b_2^* \rangle = (2)(0) + (0)(2) = 0$ — orthogonal, as required.

Looking Ahead: Where Gram–Schmidt Leads

This process reappears, in a modified “integer-preserving” form, inside the **LLL algorithm** — the classic algorithm for turning a bad lattice basis into a reasonably good one, and a core tool used both to *attack* weak lattice-based schemes and to reason about the security of well-parametrized ones. We’ll meet it properly in a later lecture; for now, just remember that Gram–Schmidt is the geometric heart of it.

Exercise: Practice

Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$: compute b_1^* , $\mu_{2,1}$, and b_2^* , and verify $\langle b_1^*, b_2^* \rangle = 0$.

8 Lattices, Properly Defined

Everything so far has allowed *any* scalar from the field F (e.g., any real number) when forming linear combinations. Informally, a lattice is what happens when we impose one dramatic restriction: **only integer combinations are allowed**, which turns a solid, continuous vector space into a discrete grid of isolated points. That informal picture (Figure 3 below) is correct and worth keeping in your head. But the definition we will actually use — the one used in serious treatments of the subject, such as Chris Peikert’s graduate lecture notes *Lattices in Cryptography*¹ — starts from the **subgroup** idea from Section 3.1, not from a basis. We will show the two views agree.

8.1 Discrete Sets

Definition: Discrete Subset

A subset $S \subseteq \mathbb{R}^n$ is **discrete** if every point of S is *isolated*: for every $x \in S$, there exists some $\varepsilon > 0$ such that the open ball $B(x, \varepsilon)$ (all points within distance ε of x) contains *no* point of S other than x itself. Informally: you can always draw a small enough circle around any point of S that catches nothing else from S .

Remark: A Shortcut for Subgroups

Checking every single point of a set for isolation sounds like a lot of work. But if H is a *subgroup* of $(\mathbb{R}^n, +)$ (Section 3.1), there’s a shortcut: H is discrete if and only if the origin $\vec{0}$ alone is isolated, i.e., there is some $\varepsilon > 0$ with $B(\vec{0}, \varepsilon) \cap H = \{\vec{0}\}$. *Why this works*: suppose $\vec{0}$ is isolated with radius ε , and suppose some other point $y \in H$ were within ε of some $x \in H$. Since H is a subgroup, $y - x \in H$ as well, and $y - x$ is within ε of $\vec{0}$ — so by our assumption, $y - x = \vec{0}$, i.e., $y = x$. So checking one point (the origin) is enough for the whole subgroup, thanks to closure under subtraction.

8.2 The Definition

Definition: Lattice

A **lattice** is a discrete additive subgroup L of $\mathbb{R}^n$. That is, $L \subseteq \mathbb{R}^n$ is a lattice if:

- (L1) L is a subgroup of $(\mathbb{R}^n, +)$ (Section 3.1): it contains $\vec{0}$, and is closed under addition and negation.
- (L2) L is discrete: by the shortcut above, there exists $\varepsilon > 0$ such that $B(\vec{0}, \varepsilon) \cap L = \{\vec{0}\}$.

¹C. Peikert, *Lattices in Cryptography*, lecture notes, <https://github.com/cpeikert/LatticesInCryptography>.

Example: $\mathbb{Q}$ Is a Subgroup, But *Not* a Lattice

Consider $\mathbb{Q} \subseteq \mathbb{R}$ (the rational numbers) under addition. This *is* a subgroup: it contains 0, the sum of two rationals is rational, and $-a$ is rational whenever a is. So condition (L1) holds. But condition (L2) fails badly: the rationals are **dense** in $\mathbb{R}$, meaning every open ball around 0, no matter how tiny, contains infinitely many other rational numbers. There is no $\varepsilon > 0$ that isolates 0. So $\mathbb{Q}$ is a perfectly good subgroup that is **not** a lattice — this is exactly why condition (L2) has to be part of the definition; subgroup alone is not a strong enough requirement.

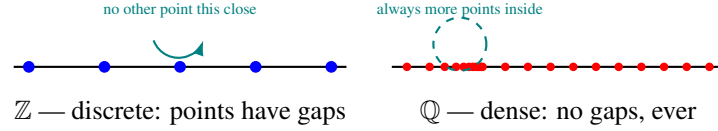


Figure 2: Discreteness, visualized. Left: in $\mathbb{Z}$, every point has a gap of size 1 around it with nothing else nearby — discrete. Right: in $\mathbb{Q}$, no matter how small a circle you draw around a point, there are always infinitely many more rational points crammed inside it — dense, and therefore *not* discrete.

Example: $\mathbb{Z}$ and $\mathbb{Z}^n$ Are Lattices

$\mathbb{Z} \subseteq \mathbb{R}$ is a subgroup (trivially — and recall from Section 3.1 that $n\mathbb{Z}$ is a subgroup of $\mathbb{Z}$ for any n ; here we’re just looking at $\mathbb{Z}$ itself sitting inside $\mathbb{R}$), and it is discrete: taking $\varepsilon = 0.5$, $B(0, 0.5) \cap \mathbb{Z} = \{0\}$, since the nearest other integers (± 1) are a full unit away. So $\mathbb{Z}$ is a lattice in $\mathbb{R}$ — the simplest possible example. The same argument, coordinate by coordinate, shows $\mathbb{Z}^n \subseteq \mathbb{R}^n$ (all integer-coordinate points) is a lattice in $\mathbb{R}^n$; it is often called **the integer lattice**.

8.3 Recovering the Basis Picture

The subgroup definition is more general and more standard, but the “grid generated by a basis” picture you may already have in your head is not wrong — it is a theorem, not a definition, and it’s worth seeing why.

Fact: Basis Representation of Lattices

Let $L \subseteq \mathbb{R}^n$. Then L is a lattice (in the sense of the subgroup definition above, i.e., a discrete additive subgroup) **if and only if** there exist linearly independent vectors $b_1, \dots, b_k \in \mathbb{R}^n$ (with $k \leq n$) such that

$$L = \mathcal{L}(b_1, \dots, b_k) = \left\{ \sum_{i=1}^k a_i b_i \mid a_i \in \mathbb{Z} \right\}.$$

The vectors $b_1, \dots, b_k$ are called a **basis** of L ; the number k is the **rank** of L . If $k = n$ (the basis spans all of $\mathbb{R}^n$), L is called **full-rank** — this is the common case in cryptography.

Remark: Proof Idea (Sketch Only)

Easy direction ($\Leftarrow$): given linearly independent $b_1, \dots, b_k$, the set of integer combinations is clearly a subgroup (adding two integer combinations, or negating one, gives another integer combination). Discreteness holds because the b_i point in genuinely independent directions: you cannot make $\sum a_i b_i$ arbitrarily close to $\vec{0}$ without every a_i being exactly 0 — linear independence rules out any “near-cancellation.” (Making this fully rigorous uses the fact that the linear map $(a_1, \dots, a_k) \mapsto \sum a_i b_i$ has a bounded inverse on its image.)

Harder direction ($\Rightarrow$): given only that L is a discrete subgroup, we must *construct* a basis. The idea is greedy and inductive: pick b_1 to be a shortest nonzero vector in L (this exists precisely *because* L is discrete — in a non-discrete set like $\mathbb{Q}$, there is no shortest nonzero element, since you can always find a smaller one). Then look at the part of L not lying on the line through b_1 , and pick b_2 to be a shortest vector there, and so on. A careful argument (using discreteness at each step, and projecting onto the space orthogonal to the vectors already chosen) shows this process terminates with a valid basis after $k \leq n$ steps. The full details are carried out rigorously in Peikert’s notes; we take the result as given here.

Looking Ahead: A Name for the “Shortest Vector”

The shortest nonzero vector used in the proof sketch above has a standard name: $\lambda_1(L)$, the **minimum distance** of the lattice. Computing $\lambda_1(L)$ — or even just estimating it — for a lattice given only a (possibly bad) basis is called the **Shortest Vector Problem (SVP)**, and it is believed to be extremely hard in high dimensions. This hardness is one of the two or three core hard problems that lattice-based cryptography’s security ultimately rests on. We’ll return to this once we reach the LWE problem.

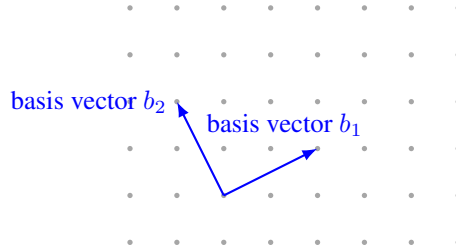


Figure 3: A 2-dimensional, full-rank lattice generated by a basis $\{b_1, b_2\}$: every grid dot is reachable by some *integer* combination $a_1 b_1 + a_2 b_2$ (with $a_1, a_2 \in \mathbb{Z}$), starting from any chosen origin dot. This is the same object as the subgroup definition above — just viewed through its basis instead of as an abstract subgroup.

Remark: Same Lattice, Different Basis

A single lattice has *many* different valid bases (just as a vector space does — recall the “Basis of $\mathbb{R}^2$ ” example in Section 7). Two bases generate the exact same set of points if and only if one can be converted to the other using an integer matrix with determinant ± 1 (a **unimodular** transformation) — we won’t need the details of this fact yet, but the consequence is critical: some bases consist of short, nearly-perpendicular vectors (a “good” basis, easy to compute with), while other, equally valid bases for the *same* lattice consist of long, nearly-parallel vectors (a “bad” basis, hard to compute with). **This gap between good and bad bases for the same lattice is the single most important idea in lattice-based cryptography:** the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad (long, scrambled-looking) basis. Recovering the good basis from the bad one is believed to be extremely hard once the dimension n is large enough.

Example: A Tiny Lattice by Hand

Let $b_1 = (2, 0)$ and $b_2 = (0, 3)$ in $\mathbb{R}^2$. The lattice $\mathcal{L}(b_1, b_2)$ consists of every point $(2a_1, 3a_2)$ for integers a_1, a_2 — so it includes $(0, 0)$, $(2, 0)$, $(-2, 0)$, $(0, 3)$, $(2, 3)$, $(4, -3)$, and so on, but *not* $(1, 1)$ or $(1, 0)$, since those cannot be written as $(2a_1, 3a_2)$ for integer a_1, a_2 . As a sanity check against the subgroup definition directly: is this set a subgroup? Yes — closed under addition and negation, contains $(0, 0)$. Is it discrete? Yes — the nearest point to the origin is at distance 2, so any $\varepsilon < 2$ isolates $\vec{0}$.

8.4 The Determinant (Volume) of a Lattice

One more piece of structure will matter a great deal in later lectures: every lattice has a well-defined “volume,” independent of which basis you use to describe it.

Definition: Fundamental Domain and Determinant

Given a basis $b_1, \dots, b_n$ of a full-rank lattice $L \subseteq \mathbb{R}^n$, the **fundamental parallelepiped** is

$$\mathcal{P}(B) = \left\{ \sum_{i=1}^n t_i b_i \mid 0 \leq t_i < 1 \right\},$$

i.e., the region “tiled out” by taking all *fractional* (not just integer) combinations between 0 and 1. Copies of $\mathcal{P}(B)$, shifted by every lattice point, tile all of $\mathbb{R}^n$ with no gaps or overlaps. The **determinant** of the lattice, $\det(L)$, is the volume of $\mathcal{P}(B)$, computed as $\det(L) = |\det(B)|$, where B is the matrix whose rows (or columns) are $b_1, \dots, b_n$.

Remark: Why $\det(L)$ Doesn’t Depend on Which Basis You Pick

Recall from the “Same Lattice, Different Basis” remark above: any two bases of the same lattice are related by a unimodular matrix U (integer entries, $\det(U) = \pm 1$), i.e., $B' = UB$. Then $\det(B') = \det(U) \det(B) = \pm \det(B)$, so $|\det(B')| = |\det(B)|$. This confirms $\det(L)$ is a genuine property of the lattice itself, not an artifact of which basis happened to be used to compute it — exactly the kind of basis-independence you should expect from something called “the volume of the lattice.”

Example: Computing $\det(L)$ for the Tiny Lattice

For $b_1 = (2, 0)$, $b_2 = (0, 3)$ from the example above, $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, so $\det(L) = |\det(B)| = |2 \times 3 - 0 \times 0| = 6$. Geometrically: the fundamental parallelepiped is just the 2×3 rectangle $[0, 2) \times [0, 3)$, and copies of this rectangle exactly tile the plane, anchored at every lattice point — matching the picture in Figure 3’s style, just for this specific (axis-aligned) lattice.

8.5 Minkowski’s First Theorem: Volume Forces a Short Vector to Exist

Everything so far has been about a *specific* lattice: given a basis, compute $\lambda_1(L)$ or $\det(L)$ directly. Minkowski’s theorem flips the question around: *without ever finding a shortest vector by hand*, can we predict, from $\det(L)$ alone, an upper bound on how long $\lambda_1(L)$ is allowed to be? Surprisingly, yes — and the argument is pure geometry, no algebra required.

Definition: Minkowski's First Theorem

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice, and let $K \subseteq \mathbb{R}^n$ be a **symmetric** ($K = -K$, i.e. $x \in K \Rightarrow -x \in K$) **convex** region. If

$$\text{Vol}(K) > 2^n \det(L),$$

then K contains a nonzero lattice point, i.e. $K \cap (L \setminus \{\vec{0}\}) \neq \emptyset$.

Remark: Reading the Theorem in Plain Language

Think of $\det(L)$ as “how much empty space, on average, surrounds each lattice point.” The theorem says: if you draw *any* symmetric, convex region big enough — bigger than 2^n times that average empty space — it is *geometrically impossible* for that region to dodge every lattice point except the origin. This is a statement about *existence only*: it doesn't tell you *which* lattice point is inside K , or how to find it — only that one is guaranteed to be there. That “existence without construction” flavor is typical of the deep facts underlying lattice cryptography, and it's worth getting comfortable with that style of guarantee now.

Remark: Proof Idea: Shrink, Overlap, and Use Symmetry (Sketch Only)

The proof is a clean pigeonhole argument, in three steps:

1. **Shrink K by half.** Let $K' = \frac{1}{2}K = \{\frac{1}{2}x : x \in K\}$. Since volume scales with the n -th power of a linear shrink factor, $\text{Vol}(K') = \text{Vol}(K)/2^n > \det(L)$.
2. **Force an overlap.** Consider translating K' by every lattice point: the copies $K' + v$ for $v \in L$. If all these copies were disjoint, then within any one fundamental domain (volume $\det(L)$, Section 8.4) the *total* volume of K' -material passing through could never exceed $\det(L)$ — but $\text{Vol}(K') > \det(L)$ already, on its own, before even counting other copies. So the copies cannot all be disjoint: some two of them, $K' + v_1$ and $K' + v_2$ with $v_1 \neq v_2$, must overlap.
3. **Use symmetry and convexity to extract a lattice point.** Overlapping means there exist $x_1, x_2 \in K'$ with $x_1 + v_1 = x_2 + v_2$, i.e. $v_1 - v_2 = x_2 - x_1$. Because K' is symmetric, $-x_1 \in K'$ too. Because K' is convex, the midpoint $\frac{1}{2}(x_2 + (-x_1)) \in K'$, which rearranges to $x_2 - x_1 \in 2K' = K$. So $v_1 - v_2 \in K$ — and $v_1 - v_2$ is a genuine *nonzero* lattice vector (since $v_1 \neq v_2$). That nonzero lattice vector is the point the theorem promised.

Nothing here required knowing where the lattice points actually are — only that K was “too big, relative to $\det(L)$,” for its translated copies to avoid colliding.

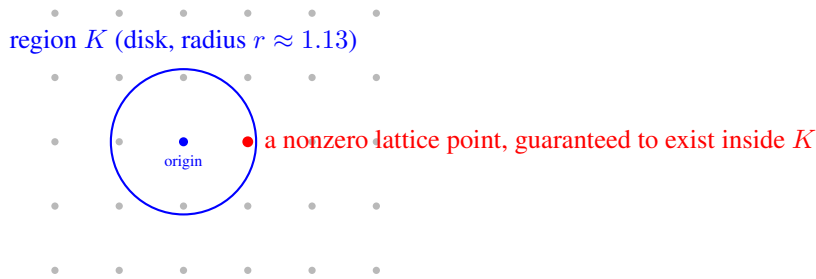


Figure 4: Minkowski's First Theorem for $L = \mathbb{Z}^2$ ($\det(L) = 1$): once the disk's area exceeds $2^2 \cdot \det(L) = 4$ (radius $r \gtrsim 1.13$), the theorem guarantees a nonzero lattice point lies inside it — and indeed $(1, 0)$ (shown here relative to the chosen origin) is inside, at distance exactly $1 < 1.13$.

Example: Minkowski's Bound on the Simplest Lattice, $L = \mathbb{Z}^2$

Take $L = \mathbb{Z}^2$, so $\det(L) = 1$ (Section 8.4). Let K be a disk of radius r centered at the origin — symmetric and convex, exactly as the theorem requires. The theorem applies once $\text{Vol}(K) = \pi r^2 > 2^2 \cdot \det(L) = 4$, i.e. once

$$r > \frac{2}{\sqrt{\pi}} \approx 1.128.$$

So Minkowski's theorem *guarantees*, without any further work, that a disk of radius 1.13 around the origin must contain a nonzero lattice point. Checking directly: we already know $\lambda_1(\mathbb{Z}^2) = 1$ (the point $(1, 0)$), and indeed $1 < 1.128$ — consistent with, and in fact comfortably inside, the guarantee. Notice the theorem didn't need to know that $(1, 0)$ was the answer; it only needed the disk to be big enough.

Example: The Same Bound on a Very Different-Looking Lattice

Now take a stretched lattice with basis $b_1 = (5, 0)$, $b_2 = (0, 1)$, so $\det(L) = |5 \times 1 - 0 \times 0| = 5$ (Section 8.4). The disk threshold becomes $\pi r^2 > 4 \times 5 = 20$, i.e. $r > \sqrt{20/\pi} \approx 2.523$. Minkowski's theorem guarantees a nonzero lattice point within distance 2.523 of the origin. The actual shortest vector here is $(0, 1)$, of length 1 — again comfortably inside the guaranteed radius, but this time the guarantee is quite *loose*: the theorem promised “within 2.523” and reality delivered “within 1.” This is expected and normal — Minkowski's theorem is a **worst-case guarantee**, not an exact formula; it never overpromises (the bound always holds), but it can certainly underdeliver on precision for any one specific lattice. Its real value is as a *universal* ceiling that holds for *every* lattice of a given determinant, not as a precise estimate for one.

Remark: From the Disk Bound to the General Formula

Redo the disk calculation above in n dimensions instead of 2: the volume of a radius- r ball in $\mathbb{R}^n$ shrinks (relative to r^n) as n grows, and pushing the same threshold argument through (we omit the n -dimensional volume formula itself, which involves the Gamma function) yields the general bound already stated in Lecture 1's introduction to this topic:

$$\lambda_1(L) \leq \sqrt{n} \cdot \det(L)^{1/n} \quad (\text{up to a small constant factor}).$$

In plain terms: a lattice can never “hide” an unexpectedly long shortest vector behind a small determinant — volume and minimum distance are locked together by geometry alone, regardless of which basis you're handed to describe the lattice. This is part of *why* SVP is only believed hard in the specific regime cryptographers deliberately choose (large dimension n , carefully scaled $\det(L)$), rather than for arbitrary lattices — a preview of the parameter-selection discussion we'll return to once we look at how Kyber and Dilithium choose their concrete dimensions and moduli.

Exercise: Practice

Let $L = \mathbb{Z}^3$ (so $\det(L) = 1$). Using the volume of a ball in $\mathbb{R}^3$, $\text{Vol} = \frac{4}{3}\pi r^3$, find the radius r at which Minkowski's theorem first guarantees a nonzero lattice point inside the ball. Compare it to the true value $\lambda_1(\mathbb{Z}^3) = 1$ (e.g. the point $(1, 0, 0)$) — is the guarantee looser or tighter, relative to the truth, than it was for $\mathbb{Z}^2$ in the worked example above?

Looking Ahead: Optional / Advanced: Minkowski's Second Theorem

The First Theorem only talks about $\lambda_1(L)$, the *single* shortest vector. It says nothing about the second-shortest independent direction, the third, and so on. **Minkowski's Second Theorem** fills that gap: define the **successive minima** $\lambda_1(L) \leq \lambda_2(L) \leq \dots \leq \lambda_n(L)$, where $\lambda_i(L)$ is the smallest radius r such that the ball of radius r contains i *linearly independent* lattice vectors. The theorem bounds their *product*:

$$\frac{2^n}{n!} \det(L) \leq \lambda_1(L) \cdots \lambda_n(L) \leq 2^n \det(L).$$

Do we need this for the thesis? Not as core material — everything in Sessions 1–6 (understanding LWE, the algorithms, and the physical attacks in Section 2 of the proposal) only ever leans on the First Theorem's intuition: “small determinant $\Rightarrow$ short vector must exist.” The Second Theorem becomes genuinely useful only if the research moves toward analyzing *trapdoor* lattices and Gaussian sampling in depth — which is exactly what underlies Falcon's signing procedure (flagged for Session 5). A trapdoor basis needs *all* of its vectors short, not just the shortest one, for Gaussian sampling to be both efficient and secure; that “all vectors short together” requirement is precisely what the successive minima $\lambda_1, \dots, \lambda_n$ capture, and the Second Theorem is the tool that ties them back to $\det(L)$. Treat this box as a pointer to come back to *if and only if* the Phase 2 research direction (Section 4 of the proposal) ends up involving Falcon's sampler or trapdoor generation in detail — otherwise, the First Theorem alone is sufficient background.

Exercise: Practice

For the lattice generated by $b_1 = (1, 2)$, $b_2 = (3, 1)$ (from the earlier homework exercise): compute $\det(L)$ directly from the basis matrix. Then pick any *other* valid basis for the same lattice (e.g., $b'_1 = b_1$, $b'_2 = b_2 - b_1$) and confirm you get the same $\det(L)$.

Looking Ahead: Where This Leads Next Lecture

With Groups $\rightarrow$ Subgroups $\rightarrow$ Rings $\rightarrow$ Fields $\rightarrow$ Modular arithmetic $\rightarrow$ Vector spaces $\rightarrow$ Basis $\rightarrow$ Lattice now in place, the next lecture can finally state the **Learning With Errors (LWE)** problem precisely: a secret vector, hidden inside noisy linear equations over $\mathbb{Z}_q$ (a ring, Section 6), which is exactly the kind of “bad basis” hardness described above, dressed up in a specific, standardized cryptographic form. We will also unpack the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ flagged in Section 4, since **module lattices** (lattices whose coordinates come from a ring like R_q instead of plain integers) are exactly what Kyber and Dilithium use in practice.

9 Summary Table

Structure	Requires	Example
Group	1 operation, inverses exist	$(\mathbb{Z}, +)$
Subgroup	A subset that is itself a group	$2\mathbb{Z} \subseteq \mathbb{Z}$
Coset / Quotient group	G sorted into shifted copies of H	$\mathbb{Z}/3\mathbb{Z} = \{0, 1, 2\}$
Ring	2 operations, no division needed	$(\mathbb{Z}, +, \times), \mathbb{Z}[x]$
Integral domain	Ring, no zero divisors	$\mathbb{Z}, \mathbb{Z}_5$ (not $\mathbb{Z}_6$)
Field	Ring + division by nonzero elements	$\mathbb{Q}, \mathbb{R}, \mathbb{Z}_p$ (p prime)
Modulus	Wraparound arithmetic	$\mathbb{Z}_n = \{0, \dots, n-1\}$
Vector space	Field + vectors + scalar mult.	$\mathbb{R}^n$
Basis	Independent, spanning vector set	$\{(1, 0), (0, 1)\}$ in $\mathbb{R}^2$
Orthogonal basis	Basis, all pairs perpendicular	Gram–Schmidt output
Lattice	Discrete additive subgroup of $\mathbb{R}^n$	$\mathbb{Z}^n$; grid of points
$\det(L)$	Volume of the fundamental domain	$ \det(B) $, any basis B

10 Full List of Exercises (Assign as Homework Before Lecture 2)

Exercise: Homework Set

1. Verify $(\{1, 2, 3, 4\}, \times \bmod 5)$ is a group by writing out its full multiplication table (Section 3 exercise, above).
2. Is $3\mathbb{Z} = \{\dots, -6, -3, 0, 3, 6, \dots\}$ a subgroup of $(\mathbb{Z}, +)$? Justify using the subgroup test (Section 3.1). Is $\{0, 3, 6, 9, \dots\}$ (only the non-negative multiples of 3) a subgroup of $(\mathbb{Z}, +)$? Why or why not?
3. List all distinct cosets of $4\mathbb{Z}$ in $\mathbb{Z}$, confirm there are exactly 4, and identify which coset -5 belongs to (Section 3.2 exercise, above).
4. Find a pair of zero divisors in $\mathbb{Z}_{12}$, and use the zero-divisor fact (Section 4.1) to explain why $\mathbb{Z}_{12}$ cannot be a field, without computing any inverses.
5. Compute $23 \times 15 \bmod 7$; list the non-invertible elements of $\mathbb{Z}_8$; use the Extended Euclidean Algorithm (Section 6.1) to find the inverse of $3 \bmod 7$ — confirm it matches the worked example.
6. Use the Extended Euclidean Algorithm to find $5^{-1} \bmod 11$ (Section 6.1 exercise, above), showing all steps.
7. Is $\mathbb{Z}_9$ a field? Why or why not? Answer it two ways: (a) using gcd (Section 6), and (b) using zero divisors (Section 4.1).
8. Show that $\{(1, 1), (1, -1)\}$ is a valid basis of $\mathbb{R}^2$ by (a) checking linear independence and (b) writing the vector $(4, 2)$ as a linear combination of them.
9. Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$ (Section 7.2 exercise, above), and verify the result is orthogonal.
10. For the lattice generated by $b_1 = (1, 2)$ and $b_2 = (3, 1)$: (a) is the point $(7, 5)$ in the lattice? (b) compute $\det(L)$ from this basis; (c) pick a different valid basis for the same lattice and confirm $\det(L)$ is unchanged (Section 8.4 exercise, above).
11. Let $L = \{(x, y) \in \mathbb{Z}^2 : x + y \text{ is even}\}$. (a) Show L is a subgroup of $(\mathbb{R}^2, +)$. (b) Show L is discrete (find an ε that isolates $\vec{0}$). (c) Since L is therefore a lattice, find a basis $\{b_1, b_2\}$ for it, and compute $\det(L)$.
12. In your own words (3–4 sentences): why does the definition of a lattice need *both* “subgroup” and “discrete”? Use the $\mathbb{Q}$ example from Section 8.2 to explain what goes wrong if you drop the discreteness requirement.
13. In your own words (3–4 sentences), explain why a “good basis” and a “bad basis” for the same lattice can make the same mathematical object feel completely different to work with computationally, now that you can measure this with norms (Section 7.1).

Lecture 2: From Lattices to Learning With Errors

SVP, CVP, SIS, LWE, Ring-LWE, and Module-LWE

Dr. Faisal Aslam

Lecture Notes — Session 2 (approx. 2 hours)

Purpose of this lecture. Lecture 1 built the algebraic toolkit — *Group* $\rightarrow$ *Subgroup* $\rightarrow$ *Ring* $\rightarrow$ *Field* $\rightarrow$ *Modular Arithmetic* $\rightarrow$ *Vector Space* $\rightarrow$ *Basis* $\rightarrow$ *Lattice* $\rightarrow$ *Determinant* — and ended by naming the Shortest Vector Problem (SVP) as one of the hard problems lattice cryptography rests on. This lecture finishes the job: we state SVP and CVP formally, meet the **Short Integer Solution (SIS)** and **Learning With Errors (LWE)** problems precisely, and then generalize LWE to **Ring-LWE** and **Module-LWE** — the exact hard problems underneath Kyber (ML-KEM) and Dilithium (ML-DSA). By the end, the public key of a real post-quantum scheme should look to you like “just an LWE sample,” not a mysterious block of numbers.

Contents

1 Quick Recap of Lecture 1	1
2 Two Canonical Hard Problems on a Lattice	2
2.1 The Shortest Vector Problem (SVP)	2
2.2 The Closest Vector Problem (CVP)	2
3 The Short Integer Solution (SIS) Problem	3
4 Learning With Errors (LWE)	3
4.1 The Idea, in Plain Language	3
4.2 Formal Definition	4
4.3 A Fully Worked Numeric Example	4
4.4 Why LWE Is Believed Hard: the Lattice Connection	5
5 From Plain LWE to Ring-LWE	5
5.1 The Problem With Plain LWE: Size	5
5.2 The Fix: Move Into a Polynomial Ring	6
5.3 Ring-LWE, Formally	6
6 Module-LWE: the Middle Ground Kyber and Dilithium Actually Use	7
7 Bridge to Session 3: A Real Public Key Is an (M)LWE Sample	8
8 Summary Table	8
9 Full List of Exercises (Assign as Homework Before Lecture 3)	8

1 Quick Recap of Lecture 1

Before building anything new, let’s fix the vocabulary we’ll be using constantly today.

Term	One-line meaning
Ring $(R, +, \times)$	Two operations; addition invertible, multiplication need not be
$\mathbb{Z}_q$	Integers $\{0, \dots, q-1\}$, wraparound (“clock”) arithmetic
Vector space, basis	Every point = a unique combination of basis directions
Lattice L	<i>Integer-only</i> combinations of a basis; a discrete grid in $\mathbb{R}^n$
$\det(L)$	Volume of the lattice’s fundamental cell; basis-independent
$\lambda_1(L)$	Length of the shortest nonzero vector in L
Good basis vs. bad basis	Short, near-orthogonal vs. long, skewed — same lattice, different difficulty

Remark: The One Idea Everything Today Builds On

Lecture 1 ended with this sentence: “*the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad basis.*” Today we make that idea concrete and computational. Every hard problem in this lecture is, underneath, a question of the form: “*given a bad description of a lattice, recover something a good description would make trivial.*”

2 Two Canonical Hard Problems on a Lattice

2.1 The Shortest Vector Problem (SVP)

Definition: Shortest Vector Problem (SVP)

Given a basis $B = \{b_1, \dots, b_n\}$ for a lattice $L = \mathcal{L}(B) \subseteq \mathbb{R}^n$, find a nonzero vector $v \in L$ with $\|v\| = \lambda_1(L)$ (the minimum distance defined in Lecture 1). The **decision** version, GapSVP_γ , only asks: is $\lambda_1(L) \leq d$, or is $\lambda_1(L) > \gamma \cdot d$, for a given distance d and approximation factor $\gamma \geq 1$? (One of these two must hold; the problem promises no “messy middle” case.)

Remark: Why the “Approximation Factor” γ Matters

Finding the *exact* shortest vector ($\gamma = 1$) is extremely hard even for classical computers once n is a few hundred. But so is finding a vector even *polynomially longer* than the shortest one, as long as γ stays modest and n is large — and that weaker, “approximate” version is all cryptography actually needs to be hard. This is why you will see phrases like “SVP is hard to approximate within polynomial factors” rather than just “SVP is hard.”

2.2 The Closest Vector Problem (CVP)

Definition: Closest Vector Problem (CVP)

Given a basis B for a lattice $L \subseteq \mathbb{R}^n$ and a **target point** $t \in \mathbb{R}^n$ (not necessarily in L), find the lattice point $v \in L$ minimizing $\|t - v\|$. The approximate decision version GapCVP_γ again just distinguishes “some lattice point within distance d of t ” from “every lattice point is farther than $\gamma \cdot d$ from t .”

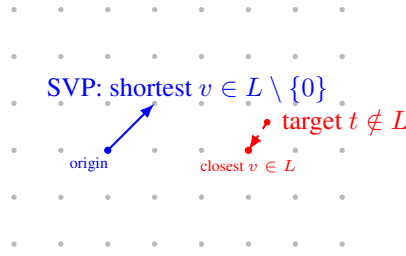


Figure 1: SVP (left, blue) asks for the shortest nonzero vector reachable inside the lattice itself. CVP (right, red) asks, for an arbitrary target point that is *not* on the grid, which grid point is nearest. Both are easy to state and easy to solve by eye in 2D — and famously hard in high dimensions with only a bad basis to work with.

Example: CVP by Eye vs. CVP by Formula

In Figure 1 you can *see* the closest grid point instantly. That’s because your eye is effectively using a very good, orthogonal basis (the obvious horizontal/vertical grid directions). Algorithmically, CVP is solved by writing $t = \sum c_i b_i$ (allowing real, non-integer c_i) and **rounding each c_i to the nearest integer** — but this shortcut only gives the *true* closest point when the basis is good (short, near-orthogonal). Hand the exact same lattice to an algorithm with a bad basis instead, and naive rounding can miss the true closest point by a wide margin. This gap — correct-with-a-good-basis, unreliable-with-a-bad-basis — is precisely what CVP-based cryptography exploits.

Looking Ahead: Where SVP and CVP Are Heading

We will not attack SVP or CVP directly today. Instead, both SIS (Section 3) and LWE (Section 4) are best understood as *specific, structured* instances of SVP/CVP, dressed up so that a random instance is easy to *generate* but — conjecturally — just as hard to *solve* as the worst case. That “random instances are as hard as the worst case” guarantee is the single most important design property of modern lattice cryptography, and we return to it in Section 4.4.

3 The Short Integer Solution (SIS) Problem

Definition: Short Integer Solution (SIS)

Fix positive integers $n, m > n$, a modulus q , and a bound β . Given a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$, find a nonzero vector $x \in \mathbb{Z}^m$ with

$$Ax \equiv \vec{0} \pmod{q} \quad \text{and} \quad \|x\| \leq \beta.$$

Remark: Why This Is Secretly a Lattice Problem

Define $\Lambda^\perp(A) = \{x \in \mathbb{Z}^m : Ax \equiv \vec{0} \pmod{q}\}$. A quick check confirms $\Lambda^\perp(A)$ is a subgroup of $\mathbb{Z}^m$ (closed under addition and negation, contains $\vec{0}$) and it is discrete (it’s a subset of $\mathbb{Z}^m$) — so by Lecture 1’s subgroup definition, $\Lambda^\perp(A)$ **is itself a lattice**. Solving SIS is exactly finding a short nonzero vector in $\Lambda^\perp(A)$ — i.e., **SIS is SVP**, restricted to this specific, algebraically-generated family of lattices. The reason cryptographers like this family is that generating a *random* instance is as easy as generating a random matrix A , yet the resulting lattice is (conjecturally) exactly as hard to solve SVP on as the worst-case lattices Lecture 1 discussed abstractly.

Example: A Tiny SIS Instance by Hand

Let $n = 1$, $m = 4$, $q = 17$, and $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix} \in \mathbb{Z}_{17}^{1 \times 4}$. We want a short, nonzero, *integer* vector $x = (x_1, x_2, x_3, x_4)$ with $3x_1 + 5x_2 + 8x_3 + x_4 \equiv 0 \pmod{17}$. Try $x = (1, 1, -1, -16) \rightarrow$ too long. Instead try $x = (1, -1, 0, 2)$: $3(1) + 5(-1) + 8(0) + 1(2) = 3 - 5 + 0 + 2 = 0 \equiv 0 \pmod{17}$. Success, and $\|x\|$ is small (each entry is ≤ 1 in absolute value except the last, which is 2). This x is a valid, short solution — with $m = 4$ columns squeezed against only $n = 1$ modular equation, short solutions are guaranteed to exist by a pigeonhole argument (more columns than rows means lots of redundancy); the point of SIS being *hard* is that finding one is nontrivial once n, m are cryptographically large (hundreds to thousands), not that one fails to exist.

Remark: What SIS Is Used For

SIS is the workhorse behind lattice-based **hash functions** and was historically the basis of early lattice signature schemes. You will meet it again properly when we discuss Dilithium/ML-DSA in Session 3 and again in Session 5, since Dilithium’s fault-attack surface is closely tied to how it uses a SIS-flavored rejection-sampling step.

4 Learning With Errors (LWE)

4.1 The Idea, in Plain Language

Imagine someone hands you a stack of linear equations in an unknown secret vector s — but each equation’s right-hand side has been nudged by a small, random amount of noise before you see it. Solving *noise-free* linear equations is easy (that’s just linear algebra — Gaussian elimination). Solving them once every equation is slightly wrong, and you don’t know by how much, turns out to be extremely hard. That is the entire idea of LWE.

4.2 Formal Definition

Definition: Learning With Errors (LWE)

Fix a dimension n , a modulus q , and an error distribution χ over $\mathbb{Z}_q$ (typically a discrete Gaussian concentrated near 0). Let $s \in \mathbb{Z}_q^n$ be a fixed secret vector.

- **Search-LWE:** given many samples (a_i, b_i) where $a_i \in \mathbb{Z}_q^n$ is uniformly random, $e_i \leftarrow \chi$ is a fresh small error, and

$$b_i = \langle a_i, s \rangle + e_i \pmod{q},$$

find s .

- **Decision-LWE:** distinguish samples (a_i, b_i) generated as above from samples where b_i is instead *uniformly random* in $\mathbb{Z}_q$, independent of a_i .

Written in matrix form for m samples stacked together: given $A \in \mathbb{Z}_q^{m \times n}$ (rows are the a_i) and $b = As + e \pmod{q}$ (with $e \leftarrow \chi^m$), search-LWE asks you to recover s .

A (public, random)	$\times$	s (secret)	$+$	e (small noise)	$=$	b (public)
known		unknown — find this		unknown, but small		known

Figure 2: The anatomy of one LWE equation $b = As + e \pmod{q}$. An attacker sees A and b (both public/known) and must recover s , without ever being told e — only that e is small. Without the noise term e , this would be ordinary linear algebra and trivial to invert; the noise is precisely what makes it hard.

4.3 A Fully Worked Numeric Example

Example: Search-LWE by Hand, $n = 3, q = 17$

Let the (unknown, to an attacker) secret be $s = (2, 9, 1) \in \mathbb{Z}_{17}^3$, and let the error distribution χ produce values in $\{-1, 0, 1\}$ only (a toy stand-in for a discrete Gaussian). Suppose four samples are generated:

a_i	$\langle a_i, s \rangle \bmod 17$	e_i	$b_i = \langle a_i, s \rangle + e_i \bmod 17$
(1, 0, 0)	2	+0	2
(0, 1, 0)	9	-1	8
(1, 1, 1)	12	+1	13
(3, 2, 1)	$6 + 18 + 1 = 25 \equiv 8$	0	8

An attacker only ever sees the a_i (left column) and b_i (right column) — *not* the middle two columns. Notice that sample 1 alone, $b_1 = 2$, looks like it might reveal $s_1 = 2$ directly — and here it does, since e_1 happened to be 0. But sample 2 gives $b_2 = 8$, while the true $s_2 = 9$: the noise $e_2 = -1$ has already shifted the visible answer away from the truth by one full step, with no way for an outside observer to know whether the “off-by-one” is coming from s_2 or from e_2 . Multiply that ambiguity across hundreds of dimensions and thousands of samples, and brute-force guessing collapses — this is the essence of why LWE is hard even though each individual equation looks almost solvable.

Exercise: Try This Before Next Lecture

Using the same A rows as above but a *different* secret $s' = (2, 8, 1)$ (differs from s only in the middle coordinate), recompute $b_1, \dots, b_4$ with the same errors e_i . Compare the resulting b_i values to the table above. How many of the four samples end up identical to the original table, purely by coincidence of the noise? What does this tell you about how many samples an attacker needs before guessing becomes hopeless?

4.4 Why LWE Is Believed Hard: the Lattice Connection

Remark: LWE Is Bounded-Distance Decoding (BDD), a Cousin of CVP

Define the lattice $\Lambda_q(A) = \{y \in \mathbb{Z}_q^m : y \equiv As \pmod{q} \text{ for some } s \in \mathbb{Z}_q^n\}$ (all vectors reachable as A times *some* secret, viewed as a lattice via the ring structure of $\mathbb{Z}_q$ from Lecture 1). The vector $b = As + e$ is, by construction, a point $As \in \Lambda_q(A)$ that has been nudged by a *small* vector e . Recovering s from b is therefore exactly **Bounded-Distance Decoding**: given a target b that is guaranteed to be *unusually close* to some lattice point (closer than a typical random point would be), find that lattice point. This is CVP’s easier, promise-bearing cousin — and, exactly as with CVP in Section 2.2, solving it reliably requires something like a good basis for $\Lambda_q(A)$. The whole point of choosing A uniformly at random is that no such good basis is visible to an attacker; only whoever *generated* s (or knows a matching trapdoor) has an edge.

Fact: Regev's Worst-Case-to-Average-Case Reduction (Statement Only)

A landmark 2005 result (Oded Regev) shows: if there existed an efficient algorithm that solves *average-case, randomly-generated* decision-LWE instances, that algorithm could be turned into an efficient algorithm for *worst-case* GapSVP and other hard lattice problems (via a quantum reduction; later work gave classical reductions for related problems). In plain terms: **LWE's hardness is not a bet made up from nothing** — it inherits its credibility from the same worst-case lattice hardness Lecture 1 built toward. This is exactly why cryptographers trust random LWE instances, generated fresh for every key, to be hard: hardness on average is tied by proof to hardness in the worst case. We take this as given; the proof itself is well beyond this course.

Looking Ahead: Search vs. Decision — Also Provably Equivalent

It's also a known theorem (not proven here) that search-LWE and decision-LWE are polynomially equivalent: an efficient solver for one can be converted into an efficient solver for the other. This matters practically because security *proofs* for encryption schemes are usually stated against decision-LWE (“ciphertexts are indistinguishable from random”), while *attacks* usually aim at search-LWE (“recover the actual secret key”). Session 5 will use both framings, so it's worth knowing they're two views of the same underlying hardness.

5 From Plain LWE to Ring-LWE

5.1 The Problem With Plain LWE: Size

A single LWE public key with security parameter n requires an $m \times n$ matrix A over $\mathbb{Z}_q$ — roughly n^2 numbers. For cryptographically secure parameters (n in the many hundreds), that's an enormous public key, and every encryption/decryption operation costs a full matrix-vector multiplication, $O(n^2)$ work. Real deployed schemes need public keys measured in kilobytes and operations fast enough for a TLS handshake or a smartcard, not this.

5.2 The Fix: Move Into a Polynomial Ring

Recall from Lecture 1's “Ring” section and its preview box: Kyber and Dilithium do arithmetic inside $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ — polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$, where powers of x wrap around using the rule $x^n \equiv -1$.

Definition: The Ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

Elements of R_q are polynomials $a(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$ with each coefficient $a_i \in \mathbb{Z}_q$. Addition is ordinary coefficient-wise addition mod q . Multiplication is ordinary polynomial multiplication, *followed by reduction*: whenever a term x^n appears, replace it with -1 (and more generally $x^{n+k} \rightarrow -x^k$), then reduce every coefficient mod q . This makes R_q a finite ring with exactly q^n elements — and, crucially, a single element of R_q packs n numbers into one object that a computer can multiply in roughly $O(n \log n)$ time using the Number-Theoretic Transform (NTT, a finite-field cousin of the FFT) — we will not need NTT details in this course, just the fact that it's the reason Ring-LWE is fast in practice.

Example: Multiplying Two Small Polynomials in R_q , $n = 4$, $q = 17$

Let $n = 4$, so we reduce using $x^4 \equiv -1$, and work mod $q = 17$. Take $a(x) = 1 + 2x$ and $c(x) = 3 + x^3$ in R_{17} . Ordinary polynomial multiplication first:

$$a(x)c(x) = (1 + 2x)(3 + x^3) = 3 + x^3 + 6x + 2x^4.$$

Now reduce using $x^4 \equiv -1$: the term $2x^4$ becomes $2 \cdot (-1) = -2$. So we're left with

$$3 + 6x + x^3 - 2 = 1 + 6x + 0x^2 + x^3.$$

Finally reduce coefficients mod 17 (here every coefficient is already in $\{0, \dots, 16\}$, so nothing changes): the product is $1 + 6x + x^3 \in R_{17}$. Notice the “wraparound” step ($x^4 \rightarrow -1$) is exactly what folds the high-degree term back down to degree < 4 — this is the polynomial-ring analogue of “carrying” in ordinary mod- q arithmetic.

5.3 Ring-LWE, Formally

Definition: Ring Learning With Errors (RLWE)

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ and an error distribution χ over R_q (coefficients drawn independently from a small distribution). Let $s \in R_q$ be a fixed secret ring element. A search-RLWE sample is a pair $(a, b) \in R_q \times R_q$ where a is uniform random and

$$b = a \cdot s + e \pmod{q}, \quad e \leftarrow \chi.$$

Search-RLWE asks for s given many such pairs; decision-RLWE asks to distinguish (a, b) from a uniformly random pair in $R_q \times R_q$.

Remark: One Ring-LWE Sample $\approx n$ Plain-LWE Samples

Compare the two definitions side by side: plain LWE needs an entire matrix $A \in \mathbb{Z}_q^{m \times n}$ to state m equations. A *single* Ring-LWE pair (a, b) , because a and s are each secretly a length- n coefficient vector multiplied together via the structured polynomial product above, is algebraically equivalent to about n ordinary LWE-style equations bundled into one ring multiplication. This is exactly where the size and speed win comes from — and exactly why the ring’s extra structure is a trade-off: it buys efficiency at the cost of a stronger (though still, so far, unbroken) hardness assumption than plain LWE, since the extra algebraic structure is, in principle, more surface area for a future attack to exploit.

6 Module-LWE: the Middle Ground Kyber and Dilithium Actually Use

Definition: Module Learning With Errors (MLWE)

Fix R_q as above and a small integer k (the **module rank**). The secret is now a length- k vector of ring elements, $\vec{s} \in R_q^k$. A sample is $(\vec{a}, b)$ with $\vec{a} \in R_q^k$ uniform random and

$$b = \langle \vec{a}, \vec{s} \rangle + e \pmod{q} = \sum_{i=1}^k a_i s_i + e, \quad e \leftarrow \chi.$$

Remark: MLWE Sits Exactly Between Plain LWE and Ring-LWE

Set $k = 1$ and MLWE collapses to Ring-LWE exactly. Let $n = 1$ (no ring structure at all — just plain integers) and MLWE collapses toward plain LWE. Choosing $k > 1$ with a modest per-ring dimension n gives designers a dial: more of the speed benefit of rings, without committing to the single largest, most algebraically rigid ring dimension. This is precisely why Kyber and Dilithium use MLWE with a moderate fixed $n = 256$ and vary k (Kyber uses $k \in \{2, 3, 4\}$ for its three security levels) instead of varying n directly — a design choice we will see in full in Session 3.

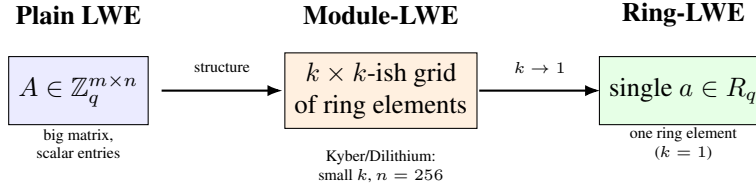


Figure 3: Module-LWE is a size/speed dial between the two extremes: no ring structure at all (plain LWE, left) and maximal ring structure (Ring-LWE, right). Kyber and Dilithium sit in the middle deliberately.

7 Bridge to Session 3: A Real Public Key *Is* an (M)LWE Sample

Looking Ahead: What You'll Recognize Next Session

When we open up Kyber (ML-KEM) in Session 3, its key generation will look almost exactly like the MLWE definition above: pick a random public matrix (of ring elements) A , pick a small secret $\vec{s}$ and small error $\vec{e}$, and publish $\mathbf{t} = A\vec{s} + \vec{e}$ as the public key — literally an MLWE sample, with $\vec{s}$ kept as the secret key. Encryption will use a *second*, independent MLWE-style sample to hide the message inside more noise. Once you can point at a real Kyber public key and say “that’s $b = As + e$,” the scheme stops looking like magic and starts looking like an application of exactly the object we defined today.

8 Summary Table

Problem	Given	Find / Decide
SVP	Basis B of lattice L	Shortest nonzero $v \in L$
CVP	Basis B , target $t \notin L$	Closest $v \in L$ to t
SIS	Random $A \in \mathbb{Z}_q^{n \times m}$	Short nonzero x : $Ax \equiv 0$
Search-LWE	$(A, b = As + e)$	Recover secret s
Decision-LWE	(A, b)	Is b “ $As + e$ ” or uniform random?
Ring-LWE	$(a, b = as + e) \in R_q^2$	Recover $s \in R_q$
Module-LWE	$(\vec{a}, b) \in R_q^k \times R_q$	Recover $\vec{s} \in R_q^k$

9 Full List of Exercises (Assign as Homework Before Lecture 3)

Exercise: Homework Set

1. State, in your own words (2–3 sentences each), the difference between SVP and CVP, and the difference between the search and decision versions of a problem in general.
2. For $n = 1$, $m = 3$, $q = 13$, $A = \begin{pmatrix} 4 & 6 & 9 \end{pmatrix}$: find a nonzero, short $x \in \mathbb{Z}^3$ solving the SIS instance $Ax \equiv 0 \pmod{13}$ (Section 3 worked example, above). Confirm your answer by direct computation.
3. Redo the search-LWE worked example (Section 4.3) with secret $s = (2, 9, 1)$ but a *new* sample $a_5 = (2, 0, 1)$ and error $e_5 = 1$. Compute b_5 . If you were only given a_5 and b_5 (not s or e_5), could you determine s from this single equation alone? Why not?
4. Explain in your own words why LWE would become *easy* (not hard) if the error e were removed entirely (i.e., $b = As$ exactly). Connect your answer to ordinary linear algebra.
5. In the ring $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$: compute the product $(2 + x^2)(1 + 3x)$, showing the wraparound step explicitly (Section 5.2 worked example, above).
6. In 3–4 sentences: why does moving from plain LWE to Ring-LWE reduce key size roughly from $O(n^2)$ numbers to $O(n)$ numbers? Use the “one Ring-LWE sample $\approx n$ plain-LWE samples” remark (Section 5.3) in your explanation.
7. Module-LWE with $k = 1$ is exactly Ring-LWE, and (informally) R_q with $n = 1$ is exactly $\mathbb{Z}_q$. Using these two facts, explain why Module-LWE is fairly described as “a generalization that contains both plain LWE and Ring-LWE as special cases.”
8. **Looking ahead:** Kyber-768 uses module rank $k = 3$ with per-ring dimension $n = 256$ and $q = 3329$. Without looking anything up yet, estimate how many total $\mathbb{Z}_q$ -coefficients are packed into one Kyber-768 secret key $\vec{s} \in R_q^3$. (You’ll check this answer in Session 3.)